

RECURSIVE CTEs

Senior Developer Mastery Guide

All Patterns · Deep Theory · 30 Graded Exercises · Full Answer Key

Every pattern from simple sequences to advanced graph algorithms

TABLE OF CONTENTS

Chapter 1 Foundations — Anatomy, Execution Model & Mental Map

Chapter 2 Pattern 1 — Number & Date Sequence Generation

Chapter 3 Pattern 2 — Hierarchical Tree Traversal

Chapter 4 Pattern 3 — Path Building & String Accumulation

Chapter 5 Pattern 4 — Aggregation Over a Hierarchy

Chapter 6 Pattern 5 — Graph Traversal & Shortest Path

Chapter 7 Pattern 6 — Transitive Closure

Chapter 8 Pattern 7 — Cycle Detection & Safety Guards

Chapter 9 Pattern 8 — Running Totals & Cumulative Aggregates

Chapter 10 Pattern 9 — Data Flattening & Unpivoting

Chapter 11 Pattern 10 — Fibonacci, Mathematical & Generative Sequences

Chapter 12 Dialect Differences: PostgreSQL vs MySQL vs SQL Server vs Oracle

Chapter 13 Performance, Anti-Patterns & Optimisation

Chapter 14 30 Graded Exercises

Chapter 15 Full Answer Key

Chapter 1 · Foundations — Anatomy, Execution Model & Mental Map

What is a Recursive CTE?

A Common Table Expression (CTE) is a named temporary result set defined with `WITH`. A recursive CTE adds the `RECURSIVE` keyword (in most dialects) and references itself in its definition. It enables SQL to express iterative processes — tree traversal, graph walking, sequence generation — that are impossible with a single `SELECT`.

Canonical Structure

◆ Recursive CTE skeleton

```
WITH RECURSIVE cte_name (col1, col2, ...) AS (

  -- ANCHOR MEMBER: runs once, seeds the recursion

  SELECT ... -- base case (non-recursive)

  UNION ALL -- or UNION to deduplicate

  -- RECURSIVE MEMBER: references cte_name

  SELECT ... -- one more step

  FROM source_table

  JOIN cte_name ON

  WHERE -- MANDATORY

)

SELECT * FROM cte_name;
```

Execution Model — Step by Step

Step 1 — Seed — The anchor member executes once. Its result set is the initial 'working table' `W(0)`.

Step 2 — Iterate — The recursive member reads from `W(i)`, produces `W(i+1)`. Rows from `W(i+1)` are appended to the result set.

Step 3 — Terminate — When the recursive member returns zero rows, iteration stops. Without a termination condition you get infinite recursion.

Step 4 — Finalise — All accumulated rows (anchor + all recursive iterations) form the final CTE result, which the outer `SELECT` queries.

UNION vs UNION ALL

Operator	Deduplicates?	Performance	Use when
UNION ALL	No	Faster	Trees, sequences, paths (no cycles possible)
UNION	Yes	Slower	Graphs with potential cycles (forces dedup)

The Three Termination Strategies

◆ Termination patterns

```
-- Strategy 1: Depth guard (cheapest)

WHERE depth < 10


-- Strategy 2: Condition on data

WHERE node.right_child IS NOT NULL


-- Strategy 3: Visited-path cycle check (graphs only)

WHERE NOT (new_node_id = ANY(visited_array))
```

■ The recursive member *MUST* reduce the problem each iteration. If your `JOIN` or `WHERE` does not guarantee progress toward termination, add a `depth < N` guard unconditionally.

When to Use a Recursive CTE

- ✓ Hierarchy / tree traversal of unknown depth
- ✓ Graph algorithms (BFS, shortest path, reachability)
- ✓ Generating sequences without a numbers table
- ✓ Accumulating running values through ordered rows
- ✓ Unpivoting / flattening variable-length data
- ✓ Mathematical series (Fibonacci, factorials, primes)

Chapter 2 · Pattern 1 — Number & Date Sequence Generation

Generating a sequence without a pre-built numbers table is one of the most practical day-to-day uses of recursive CTEs. Master this and you unlock calendar tables, test data generation, and gap-filling on the fly.

2.1 Integer Sequence 1 to N

```
WITH RECURSIVE nums AS (  
  
  SELECT 1 AS n -- anchor  
  
  UNION ALL  
  
  SELECT n + 1 -- recursive step  
  
  FROM nums  
  
  WHERE n < 100 -- termination  
  
)  
  
SELECT n FROM nums;
```

2.2 Date Series (Calendar Table)

```
WITH RECURSIVE cal AS (  
  
  SELECT CAST('2024-01-01' AS DATE) AS dt  
  
  UNION ALL  
  
  SELECT dt + INTERVAL '1 day'  
  
  FROM cal  
  
  WHERE dt < '2024-12-31'  
  
)  
  
SELECT dt,  
  
  EXTRACT(DOW FROM dt) AS day_of_week,  
  
  EXTRACT(WEEK FROM dt) AS iso_week  
  
FROM cal;
```

2.3 Fill Gaps in a Time Series

```
-- Sales table may have missing dates. Fill every date with 0 if no sale.

WITH RECURSIVE cal AS (

  SELECT MIN(sale_date) AS dt FROM sales

  UNION ALL

  SELECT dt + INTERVAL '1 day'

  FROM cal

  WHERE dt < (SELECT MAX(sale_date) FROM sales)

)

SELECT cal.dt,

  COALESCE(SUM(s.amount), 0) AS daily_total

FROM cal

LEFT JOIN sales AS s ON s.sale_date = cal.dt

GROUP BY cal.dt

ORDER BY cal.dt;
```

2.4 Business Days Only

```
WITH RECURSIVE biz_days AS (

  SELECT CAST('2024-01-01' AS DATE) AS dt, 0 AS bday_num

  UNION ALL

  SELECT

    CASE EXTRACT(DOW FROM dt + INTERVAL '1 day')

      WHEN 0 THEN dt + INTERVAL '3 days' -- Sunday -> Wednesday

      WHEN 6 THEN dt + INTERVAL '2 days' -- Saturday-> Monday

      ELSE dt + INTERVAL '1 day'
```

```
END,  
  
bday_num + 1  
  
FROM biz_days  
  
WHERE bday_num < 260 -- ~52 weeks of biz days  
  
)  
  
SELECT dt, bday_num FROM biz_days;
```

■ *In PostgreSQL you can also use `generate_series()`. The recursive CTE approach is portable across all SQL dialects.*

Chapter 3 · Pattern 2 — Hierarchical Tree Traversal

Schema — employees (adjacency list)

```
CREATE TABLE employees (

  emp_id INT PRIMARY KEY,

  name VARCHAR(100),

  dept VARCHAR(50),

  salary DECIMAL(10,2),

  manager_id INT REFERENCES employees(emp_id),

  hire_date DATE

);
```

emp_id	name	dept	salary	manager_id
1	Sarah Chen	Eng	180000	NULL
2	James Park	Eng	140000	1
3	Maria Lopez	Eng	130000	1
4	David Kim	Mkt	120000	NULL
5	Emma Wilson	Mkt	110000	4
6	Liam Brown	Eng	95000	2
7	Aisha Osei	HR	105000	NULL

3.1 Top-Down: Root to All Descendants

```
WITH RECURSIVE org AS (

  -- Anchor: roots (no manager)

  SELECT emp_id, name, manager_id,

  0 AS depth,

  name AS path
```

```

FROM employees

WHERE manager_id IS NULL

UNION ALL

-- Recursive: extend one level

SELECT e.emp_id, e.name, e.manager_id,

o.depth + 1,

o.path || ' > ' || e.name

FROM employees e

JOIN org o ON e.manager_id = o.emp_id

)

SELECT depth,

REPEAT(' ', depth) || name AS indented_name,

path

FROM org

ORDER BY path;

```

3.2 Bottom-Up: Leaf to All Ancestors

```

WITH RECURSIVE ancestry AS (

SELECT emp_id, name, manager_id, 0 AS level

FROM employees

WHERE name = 'Liam Brown' -- start from a specific leaf

UNION ALL

SELECT p.emp_id, p.name, p.manager_id, a.level + 1

```

```

FROM employees p

JOIN ancestry a ON p.emp_id = a.manager_id

)

SELECT level, name FROM ancestry ORDER BY level;

```

3.3 Subtree of a Specific Node

```

-- Everything under James Park (emp_id = 2), including James himself

WITH RECURSIVE subtree AS (

SELECT emp_id, name, manager_id, 0 AS depth

FROM employees

WHERE emp_id = 2

UNION ALL

SELECT e.emp_id, e.name, e.manager_id, s.depth + 1

FROM employees e

JOIN subtree s ON e.manager_id = s.emp_id

)

SELECT depth, name FROM subtree ORDER BY depth, name;

```

3.4 Siblings at the Same Level

```

-- Find all employees at depth 2 (grandchildren of roots)

WITH RECURSIVE org AS (

SELECT emp_id, name, manager_id, 0 AS depth

FROM employees WHERE manager_id IS NULL

UNION ALL

SELECT e.emp_id, e.name, e.manager_id, o.depth + 1

```

```
FROM employees e JOIN org o ON e.manager_id = o.emp_id

)

SELECT name FROM org WHERE depth = 2;
```

Chapter 4 · Pattern 3 — Path Building & String Accumulation

String accumulation through recursion is powerful but needs careful handling to avoid exceeding VARCHAR limits and to remain portable.

4.1 Full Path String (Root to Node)

```
WITH RECURSIVE paths AS (

  SELECT emp_id, name, manager_id,

  CAST(name AS VARCHAR(2000)) AS full_path

  FROM employees WHERE manager_id IS NULL

  UNION ALL

  SELECT e.emp_id, e.name, e.manager_id,

  CAST(p.full_path || ' / ' || e.name AS VARCHAR(2000))

  FROM employees e

  JOIN paths p ON e.manager_id = p.emp_id

)

SELECT name, full_path FROM paths ORDER BY full_path;
```

4.2 ID Path Array (PostgreSQL)

```
WITH RECURSIVE id_paths AS (

  SELECT emp_id, name, manager_id,

  ARRAY[emp_id] AS id_path

  FROM employees WHERE manager_id IS NULL

  UNION ALL

  SELECT e.emp_id, e.name, e.manager_id,
```

```

ip.id_path || e.emp_id

FROM employees e

JOIN id_paths ip ON e.manager_id = ip.emp_id

)

SELECT name, id_path FROM id_paths ORDER BY id_path;

```

DIALECT DIFFERENCES

PostgreSQL: ARRAY[emp_id] || emp_id for array accumulation

MySQL 8+: Use GROUP_CONCAT workaround – MySQL lacks array accumulation in CTEs

SQL Server: Use STRING_AGG or FOR XML PATH for path concatenation

Oracle: Use SYS_CONNECT_BY_PATH (CONNECT BY is Oracle's native hierarchy)

4.3 Materialised Path Pattern

A materialised path stores the ancestor chain as a string in each row (e.g. '1/4/5/'). Recursive CTEs can both build and query materialised paths.

```

-- Build materialised path for all nodes

WITH RECURSIVE mat_path AS (

SELECT emp_id, name, manager_id,

'/' || emp_id || '/' AS mpath

FROM employees WHERE manager_id IS NULL

UNION ALL

SELECT e.emp_id, e.name, e.manager_id,

mp.mpath || e.emp_id || '/'

FROM employees e

JOIN mat_path mp ON e.manager_id = mp.emp_id

)

```

```
SELECT name, mpath FROM mat_path;
```

```
-- Query: all descendants of node 1 (using LIKE on mpath)
```

```
SELECT name FROM mat_path WHERE mpath LIKE '/1/%';
```

Chapter 5 · Pattern 4 — Aggregation Over a Hierarchy

Rolling up values through a tree — total headcount, total salary budget, sum of all subordinate costs — requires accumulating values bottom-up or top-down through the recursive CTE.

5.1 Total Salary Budget per Manager (Subtree Sum)

```
WITH RECURSIVE subtree AS (  
  
  -- Anchor: every employee seeds their own subtree  
  
  SELECT emp_id AS root_id, emp_id, salary  
  
  FROM employees  
  
  UNION ALL  
  
  -- Recursive: pull direct reports into the root's subtree  
  
  SELECT s.root_id, e.emp_id, e.salary  
  
  FROM employees e  
  
  JOIN subtree s ON e.manager_id = s.emp_id  
  
)  
  
SELECT root_id AS manager_id,  
  
m.name AS manager_name,  
  
SUM(salary) AS total_salary_budget,  
  
COUNT(*) AS headcount  
  
FROM subtree s  
  
JOIN employees m ON m.emp_id = s.root_id  
  
GROUP BY root_id, m.name  
  
ORDER BY total_salary_budget DESC;
```

5.2 Depth-Weighted Aggregation

```

-- Weight salary by depth (closer reports count more)

WITH RECURSIVE weighted AS (

SELECT emp_id, emp_id AS root_id, salary, 1.0 AS weight

FROM employees

UNION ALL

SELECT e.emp_id, w.root_id, e.salary, w.weight * 0.5

FROM employees e

JOIN weighted w ON e.manager_id = w.emp_id

WHERE w.weight > 0.01 -- prune tiny weights

)

SELECT root_id,

SUM(salary * weight) / SUM(weight) AS weighted_avg_salary

FROM weighted

GROUP BY root_id;

```

5.3 Level Counts per Manager

```

WITH RECURSIVE org AS (

SELECT emp_id, manager_id, 0 AS depth

FROM employees WHERE manager_id IS NULL

UNION ALL

SELECT e.emp_id, e.manager_id, o.depth + 1

FROM employees e JOIN org o ON e.manager_id = o.emp_id

)

SELECT depth,

COUNT(*) AS employees_at_level,

```

```
AVG(e.salary) AS avg_salary  
  
FROM org  
  
JOIN employees e USING (emp_id)  
  
GROUP BY depth  
  
ORDER BY depth;
```

Chapter 6 · Pattern 5 — Graph Traversal & Shortest Path

Graphs differ from trees: nodes can have multiple parents and cycles are possible. Recursive CTEs implement BFS (breadth-first search) naturally, giving shortest-path semantics when using UNION.

Schema — roads (weighted graph)

```
CREATE TABLE roads (
  from_city VARCHAR(50),
  to_city VARCHAR(50),
  distance INT
);
```

from_city	to_city	distance
Dubai	Abu Dhabi	150
Dubai	Sharjah	30
Sharjah	Fujairah	120
Abu Dhabi	Al Ain	160
Al Ain	Fujairah	130
Dubai	Al Ain	150

6.1 All Reachable Cities from Dubai

```
WITH RECURSIVE reachable AS (
  SELECT to_city AS city, distance AS total_dist, 1 AS hops
  FROM roads WHERE from_city = 'Dubai'

  UNION ALL

  SELECT r.to_city, rc.total_dist + r.distance, rc.hops + 1
  FROM roads r
  JOIN reachable rc ON r.from_city = rc.city
```

```

WHERE rc.hops < 5 -- depth guard

)

SELECT city, MIN(total_dist) AS shortest_dist, MIN(hops) AS min_hops

FROM reachable

GROUP BY city

ORDER BY shortest_dist;

```

6.2 Dijkstra-Style Shortest Path (with path)

```

WITH RECURSIVE sp AS (

SELECT 'Dubai' AS city,

0 AS total_dist,

CAST('Dubai' AS VARCHAR(2000)) AS path

UNION ALL

SELECT r.to_city,

sp.total_dist + r.distance,

sp.path || ' -> ' || r.to_city

FROM roads r

JOIN sp ON r.from_city = sp.city

WHERE sp.path NOT LIKE '%' || r.to_city || '%' -- cycle guard

)

SELECT city,

MIN(total_dist) AS shortest_dist,

-- path for the min-cost route:

(SELECT path FROM sp s2

```

```
WHERE s2.city = sp.city

ORDER BY total_dist LIMIT 1) AS route

FROM sp

GROUP BY city

ORDER BY shortest_dist;
```

■ *LIKE '%city%' path check is $O(\text{path_length})$. In PostgreSQL prefer `ARRAY contains: NOT (city = ANY(path_array))`.*

Chapter 7 · Pattern 6 — Transitive Closure

Transitive closure computes ALL pairs (A, B) such that a path exists from A to B, regardless of how many hops. Use cases: dependency resolution, access-control propagation, social graph reachability.

Schema — depends_on (package dependencies)

```
CREATE TABLE depends_on (

package VARCHAR(50),

requires VARCHAR(50)

);

-- app -> lib_a -> lib_b -> lib_c (chain)

INSERT INTO depends_on VALUES

('app', 'lib_a'), ('lib_a', 'lib_b'), ('lib_b', 'lib_c'),

('app', 'lib_d'), ('lib_d', 'lib_c');
```

7.1 All Transitive Dependencies

```
WITH RECURSIVE closure AS (

-- Direct dependencies

SELECT package AS root, requires AS dep, 1 AS hops

FROM depends_on

UNION

-- Transitive: root -> intermediate -> dep

SELECT c.root, d.requires, c.hops + 1

FROM closure c

JOIN depends_on d ON d.package = c.dep

WHERE c.hops < 20 -- safety
```

```

)

SELECT root, dep, MIN(hops) AS min_hops

FROM closure

GROUP BY root, dep

ORDER BY root, min_hops;

```

■ Use *UNION* (not *UNION ALL*) here — it deduplicates naturally and prevents infinite expansion on diamond-shaped dependency graphs.

7.2 Impact Analysis — What Breaks if lib_b Changes?

```

WITH RECURSIVE closure AS (

  SELECT package AS root, requires AS dep, 1 AS hops

  FROM depends_on

  UNION

  SELECT c.root, d.requires, c.hops + 1

  FROM closure c JOIN depends_on d ON d.package = c.dep

  WHERE c.hops < 20

)

SELECT DISTINCT root AS affected_package

FROM closure

WHERE dep = 'lib_b'

ORDER BY root;

```

7.3 Counting Unique Dependencies Per Package

```

WITH RECURSIVE closure AS (

  SELECT package AS root, requires AS dep

  FROM depends_on

  UNION

```

```
SELECT c.root, d.requires

FROM closure c JOIN depends_on d ON d.package = c.dep

)

SELECT root, COUNT(DISTINCT dep) AS total_deps

FROM closure

GROUP BY root

ORDER BY total_deps DESC;
```

Chapter 8 · Pattern 7 — Cycle Detection & Safety Guards

Cycles in recursive CTEs cause infinite loops and crashed queries. Every production recursive CTE against a graph (not a pure tree) MUST include a cycle-detection mechanism.

8.1 Array-Based Cycle Detection (PostgreSQL)

```
WITH RECURSIVE safe_graph AS (

  SELECT from_city AS city,

  ARRAY[from_city] AS visited,

  false AS cycle

  FROM roads WHERE from_city = 'Dubai'

  UNION ALL

  SELECT r.to_city,

  sg.visited || r.to_city,

  r.to_city = ANY(sg.visited)

  FROM roads r

  JOIN safe_graph sg ON r.from_city = sg.city

  WHERE NOT sg.cycle

)

SELECT city, visited, cycle FROM safe_graph;

-- Report only detected cycles

SELECT city, visited FROM safe_graph WHERE cycle;
```

8.2 String-Path Cycle Check (Portable)

```
WITH RECURSIVE safe_graph AS (
```

```

SELECT from_city AS city,

CAST('' || from_city || '' AS VARCHAR(4000)) AS visited_str

FROM roads WHERE from_city = 'Dubai'

UNION ALL

SELECT r.to_city,

sg.visited_str || r.to_city || ''

FROM roads r

JOIN safe_graph sg ON r.from_city = sg.city

WHERE sg.visited_str NOT LIKE '%|' || r.to_city || '|%'

)

SELECT city, visited_str FROM safe_graph;

```

8.3 Depth-Cap Guard (Universal)

```

-- Simplest guard – always safe but may terminate early on deep graphs

WITH RECURSIVE traversal AS (

SELECT node_id, 0 AS depth FROM graph WHERE node_id = 1

UNION ALL

SELECT e.to_node, t.depth + 1

FROM edges e

JOIN traversal t ON e.from_node = t.node_id

WHERE t.depth < 50 -- hard ceiling

)

SELECT * FROM traversal;

```

■ **NEVER** run a recursive CTE on a graph without at least one of: (1) depth < N guard, (2) UNION deduplication, (3) explicit visited-path check. Missing these will hang your database session.

Chapter 9 · Pattern 8 — Running Totals & Cumulative Aggregates

Recursive CTEs can compute running values by carrying state from one row to the next — useful when window functions are unavailable or when the accumulation logic is conditional (not a simple SUM).

Schema — ledger

```
CREATE TABLE ledger (  
  
  txn_id INT PRIMARY KEY,  
  
  txn_date DATE,  
  
  amount DECIMAL(12,2) -- positive=credit, negative=debit  
  
);
```

9.1 Running Balance

```
WITH RECURSIVE running AS (  
  
  -- Anchor: first row  
  
  SELECT txn_id, txn_date, amount, amount AS balance  
  
  FROM ledger  
  
  WHERE txn_id = (SELECT MIN(txn_id) FROM ledger)  
  
  UNION ALL  
  
  -- Add next row by txn_id  
  
  SELECT l.txn_id, l.txn_date, l.amount,  
  
  r.balance + l.amount  
  
  FROM ledger l  
  
  JOIN running r ON l.txn_id = (  
  
  SELECT MIN(txn_id) FROM ledger
```

```

WHERE txn_id > r.txn_id

)

)

SELECT txn_id, txn_date, amount, balance

FROM running ORDER BY txn_id;

```

■ The correlated subquery to find 'next row' is expensive. Prefer `ROW_NUMBER()` + window `SUM` when possible. Use this pattern only when carry-forward logic is conditional.

9.2 Conditional Carry-Forward (e.g. Waterfall Allocation)

```

-- Allocate payments greedily to invoices in order

-- payment_pool carries remaining balance forward

WITH RECURSIVE alloc AS (

SELECT invoice_id, due_amount,

LEAST(due_amount, 10000.00) AS paid,

10000.00 - LEAST(due_amount, 10000.00) AS remaining_pool

FROM invoices WHERE invoice_id = (SELECT MIN(invoice_id) FROM invoices)

UNION ALL

SELECT i.invoice_id, i.due_amount,

LEAST(i.due_amount, a.remaining_pool),

a.remaining_pool - LEAST(i.due_amount, a.remaining_pool)

FROM invoices i

JOIN alloc a ON i.invoice_id = (

SELECT MIN(invoice_id) FROM invoices WHERE invoice_id > a.invoice_id

)

WHERE a.remaining_pool > 0

```

```
)
```

```
SELECT * FROM alloc;
```

Chapter 10 · Pattern 9 — Data Flattening & Unpivoting

When data is stored in JSON arrays, comma-separated strings, or variable-length encoded columns, recursive CTEs can split and flatten it without procedural code.

10.1 Split Comma-Separated Values

```
-- Table: products with comma-separated tag strings

-- product_id | tags

-- 1 | 'sql,database,performance'

WITH RECURSIVE split AS (

  SELECT product_id,

    tags || ',' AS remaining,

    '' AS tag

  FROM products

  UNION ALL

  SELECT product_id,

    SUBSTRING(remaining FROM POSITION(',') IN remaining) + 1),

    SUBSTRING(remaining FOR POSITION(',') IN remaining) - 1)

  FROM split

  WHERE remaining <> ''

)

SELECT product_id, tag

FROM split

WHERE tag <> ''
```

```
ORDER BY product_id, tag;
```

10.2 Unpivot N Columns into Rows

```
-- q1_sales, q2_sales, q3_sales, q4_sales -> one row per quarter

WITH cols(quarter_num) AS (

SELECT 1 UNION ALL SELECT 2 UNION ALL SELECT 3 UNION ALL SELECT 4

)

SELECT p.product_id,

c.quarter_num,

CASE c.quarter_num

WHEN 1 THEN p.q1_sales

WHEN 2 THEN p.q2_sales

WHEN 3 THEN p.q3_sales

WHEN 4 THEN p.q4_sales

END AS sales

FROM products p

CROSS JOIN cols c

ORDER BY p.product_id, c.quarter_num;
```

10.3 Expand Ranges into Individual Rows

```
-- Table: discounts with start_rank and end_rank

-- Expand: rank 3 to 7 -> rows for 3,4,5,6,7

WITH RECURSIVE expand AS (

SELECT discount_id, start_rank AS rank_val, end_rank, discount_pct

FROM discounts

UNION ALL
```

```
SELECT discount_id, rank_val + 1, end_rank, discount_pct

FROM expand

WHERE rank_val < end_rank

)

SELECT discount_id, rank_val, discount_pct

FROM expand ORDER BY discount_id, rank_val;
```

Chapter 11 · Pattern 10 — Fibonacci, Mathematical & Generative Sequences

11.1 Fibonacci Sequence

```
WITH RECURSIVE fib AS (  
  
  SELECT 0 AS a, 1 AS b  
  
  UNION ALL  
  
  SELECT b, a + b  
  
  FROM fib  
  
  WHERE b < 1000000  
  
)  
  
SELECT a AS fibonacci_number FROM fib;
```

11.2 Factorial

```
WITH RECURSIVE fact AS (  
  
  SELECT 1 AS n, 1 AS factorial  
  
  UNION ALL  
  
  SELECT n + 1, factorial * (n + 1)  
  
  FROM fact  
  
  WHERE n < 20  
  
)  
  
SELECT n, factorial FROM fact;
```

11.3 Prime Numbers (Sieve of Eratosthenes in SQL)

```
WITH RECURSIVE  
  
candidates AS (  
  
  SELECT 2 AS n
```

```

UNION ALL

SELECT n + 1 FROM candidates WHERE n < 100

),

primes AS (

SELECT n FROM candidates

WHERE NOT EXISTS (

SELECT 1 FROM candidates AS d

WHERE d.n BETWEEN 2 AND FLOOR(SQRT(candidates.n))

AND candidates.n % d.n = 0

)

)

SELECT n AS prime FROM primes ORDER BY n;

```

11.4 Power Set / All Subsets

```

-- Generate all subsets of {1,2,3,4}

WITH RECURSIVE

elements(val) AS (SELECT 1 UNION ALL SELECT 2 UNION ALL SELECT 3 UNION ALL SELECT 4),

subsets AS (

SELECT CAST('' AS VARCHAR(100)) AS subset, 0 AS last_val

UNION ALL

SELECT

CASE WHEN subset = '' THEN CAST(e.val AS VARCHAR)

ELSE subset || ',' || CAST(e.val AS VARCHAR) END,

e.val

FROM subsets s

```

```
JOIN elements e ON e.val > s.last_val

)

SELECT subset FROM subsets ORDER BY LENGTH(subset), subset;
```

Chapter 12 · Dialect Differences: PostgreSQL vs MySQL vs SQL Server vs Oracle

Feature	PostgreSQL	MySQL 8+	SQL Server	Oracle
Keyword	WITH RECURSIVE	WITH RECURSIVE	WITH (no keyword)	WITH (no keyword)
Max depth	Configurable	Configurable	100 (default)	No limit (use guard)
Array support	ARRAY[]	No	No	No
String concat		CONCAT()	+ or CONCAT	or CONCAT
Cycle clause	CYCLE col SET...	No	No	CYCLE (12c+)
SEARCH clause	SEARCH DEPTH/BREADTH FIRST	NO FIRST	No	CONNECT BY (native)
String split	regexp_split_to_table	SUBSTRING+LOCATE	STRING_SPLIT	REGEXP_SUBSTR
Date add	dt + INTERVAL '1 day'	dt + INTERVAL 1 DAY	DATEADD(day,1,dt)	dt + 1

PostgreSQL — Advanced Features

◆ PostgreSQL 14+ CYCLE clause

```
-- PostgreSQL 14+ native CYCLE clause (no manual array needed)

WITH RECURSIVE graph AS (

  SELECT from_city AS city, 0 AS dist

  FROM roads WHERE from_city = 'Dubai'

  UNION ALL

  SELECT r.to_city, g.dist + r.distance

  FROM roads r JOIN graph g ON r.from_city = g.city

)

CYCLE city SET is_cycle USING path

SELECT city, dist, is_cycle FROM graph WHERE NOT is_cycle;
```

◆ PostgreSQL 14+ SEARCH clause

```
-- SEARCH clause for explicit BFS / DFS ordering

WITH RECURSIVE org AS (
```

```

SELECT emp_id, name, manager_id FROM employees WHERE manager_id IS NULL

UNION ALL

SELECT e.emp_id, e.name, e.manager_id

FROM employees e JOIN org ON e.manager_id = org.emp_id

)

SEARCH DEPTH FIRST BY emp_id SET ordercol

SELECT name FROM org ORDER BY ordercol;

```

Oracle — CONNECT BY (Native Hierarchy)

◆ Oracle CONNECT BY

```

-- Oracle's proprietary hierarchy syntax (pre-dates recursive CTEs)

SELECT LEVEL,

SYS_CONNECT_BY_PATH(name, '/') AS path,

name

FROM employees

START WITH manager_id IS NULL

CONNECT BY PRIOR emp_id = manager_id;

```

SQL Server — Recursive CTE Syntax

◆ SQL Server

```

-- SQL Server uses WITH without RECURSIVE keyword

-- Default max recursion = 100; override with OPTION

WITH org AS (

SELECT emp_id, name, manager_id, 0 AS depth

FROM employees WHERE manager_id IS NULL

UNION ALL

```

```
SELECT e.emp_id, e.name, e.manager_id, o.depth + 1

FROM employees e JOIN org o ON e.manager_id = o.emp_id

)

SELECT * FROM org

OPTION (MAXRECURSION 500); -- raise recursion limit
```

Chapter 13 · Performance, Anti-Patterns & Optimisation

13.1 Performance Characteristics

Pattern	Complexity	Risk	Optimisation
Sequence gen (1..N)	$O(N)$	Low	Index on join col
Tree traversal	$O(N)$	Low	Index on parent_id
Graph BFS (cycle-free)	$O(V+E)$	Medium	UNION dedup
Transitive closure	$O(V^2)$	High	Depth cap + UNION
Running total via CTE	$O(N^2)$	High	Use window SUM()
Shortest path	$O(V * E)$	High	Cap depth, materialise
String split (CSV)	$O(N * L)$	Medium	Normalise schema instead

13.2 Indexing Checklist

- ✓ parent_id / manager_id — the most critical index for hierarchical CTEs
- ✓ from_node / from_city — join key in recursive member for graph CTEs
- ✓ Composite (parent_id, child_id) — covers both sides of the join
- ✓ Filtered index on root nodes: WHERE manager_id IS NULL
- ✓ EXPLAIN / EXPLAIN ANALYZE to verify index use in the recursive member

13.3 Anti-Patterns to Avoid

✗ No termination condition

→ Infinite loop. The database will hit its recursion limit and error, or run forever.

✗ UNION ALL on a cyclic graph

→ Rows multiply exponentially. Always use UNION on graphs, UNION ALL only on pure trees.

✗ SELECT * in recursive member

→ Carries every column through each iteration — bloats working table. Select only what you need.

✗ Running total via recursive CTE

→ $O(N^2)$. Use SUM() OVER (ORDER BY ...) instead — it's $O(N)$ and far more readable.

✗ String path check LIKE '%x%' on long paths

→ $O(\text{path_length})$ per comparison. Use ARRAY and = ANY() in PostgreSQL instead.

✗ Recursive CTE for known fixed depth

→ Unnecessary complexity. Two or three self joins are clearer and faster at fixed depths.

✗ Forgetting CAST on anchor string columns

→ Type mismatch errors as string grows through iterations. Always CAST anchor to max width.

13.4 Decision: Recursive CTE vs Alternative

Need to traverse a hierarchy?

- Known fixed depth (1-3 levels) → Multi-alias self join (simpler, faster)
- Unknown/variable depth → Recursive CTE

Need to generate a sequence?

- PostgreSQL → `generate_series()` (fastest)
- Other dialects → Recursive CTE number generator
- Very large N (>100k) → Pre-built numbers table

Need to walk a graph?

- Simple reachability → Recursive CTE + UNION
- Shortest path (simple) → Recursive CTE + MIN aggregate
- Complex graph algorithms → Dedicated graph DB (Neo4j, etc.)

Need a running total?

- Standard accumulation → Window `SUM()` OVER (ORDER BY)
- Conditional carry-forward logic → Recursive CTE (last resort)

Chapter 14 · 30 Graded Exercises

Three tiers — beginner, intermediate, advanced. Attempt each tier fully before checking Chapter 15.

BEGINNER (Exercises 1–10)

Ex 1. Using a recursive CTE, generate the integers from 1 to 50. Return the number and its square.

Ex 2. Generate every date in January 2025 using a recursive CTE.

Ex 3. Write a recursive CTE that lists all employees in the org chart, showing their name and depth (root = depth 0).

Ex 4. Find all direct and indirect reports of Sarah Chen (emp_id = 1) using a recursive CTE.

Ex 5. Write a recursive CTE that returns the path from root to each employee as a slash-separated string e.g. 'Sarah Chen / James Park / Liam Brown'.

Ex 6. Generate the first 20 Fibonacci numbers using a recursive CTE.

Ex 7. Using the employees table, write a recursive CTE to find all ancestors (managers above) of Liam Brown, from direct manager up to the root.

Ex 8. Generate the multiplication table for 7 (7x1 through 7x12) using a recursive CTE.

Ex 9. Using a recursive CTE, return all employees at exactly depth 2 in the org chart.

Ex 10. Write a recursive CTE to list all weekdays (Mon–Fri) in Q1 2025 (Jan 1 – Mar 31).

INTERMEDIATE (Exercises 11–20)

Ex 11. Compute the total salary budget for each manager's entire subtree (sum all salaries of all direct and indirect reports plus themselves).

Ex 12. Write a recursive CTE on the roads table to find the shortest distance from Dubai to every other city.

Ex 13. Split the comma-separated string 'alpha,beta,gamma,delta,epsilon' into one row per element using a recursive CTE (no built-in split functions).

Ex 14. For the depends_on table, find the full transitive closure: every (package, transitive_dependency) pair.

Ex 15. Using a recursive CTE, expand each employee's hire_date to show every month from their hire_date to '2025-12-01', giving a 'tenure calendar'.

Ex 16. Find all packages that would be affected (directly or transitively) if lib_a were removed.

Ex 17. Generate a calendar for 2025 with columns: date, day_name, week_number, is_weekend.

Ex 18. Using the ledger schema, compute a running balance from the first transaction to the last using a recursive CTE.

Ex 19. Write a recursive CTE to find all employees who are exactly 3 levels below any root node.

Ex 20. Using the roads table, find the shortest path string (not just the distance) from Dubai to Fujairah.

ADVANCED (Exercises 21–30)

Ex 21. Write a recursive CTE on the employees table that detects any cycles (an employee who is, directly or indirectly, their own manager). Return the emp_id and the cycle path.

Ex 22. Implement a breadth-first search on the roads graph from Dubai. Return city, hop count, and shortest distance, ensuring each city appears only once with its optimal values.

Ex 23. Generate all prime numbers up to 200 using only recursive CTEs (no built-in prime functions or procedural code).

Ex 24. Compute the 'org weight' of each node: defined as the sum of $(1 / 2^{\text{depth}})$ for all nodes in its subtree, where depth is relative to that node as root. Managers with more immediate reports weigh more.

Ex 25. Using the employees table, produce a complete org-chart report in a single recursive CTE that includes: name, depth, path, total subtree headcount, total subtree salary, and dense rank by salary within their depth level.

Ex 26. Given a table of project tasks with (task_id, task_name, depends_on_task_id), produce a valid topological sort order using a recursive CTE.

Ex 27. Write a recursive CTE that enumerates all possible routes from Dubai to Al Ain in the roads table, showing each unique route and its total distance, with a cycle guard to prevent infinite loops.

Ex 28. Using the ledger table, implement a 3-period exponential moving average (EMA) where $\text{EMA}(t) = \text{amount}(t) * k + \text{EMA}(t-1) * (1-k)$, $k = 2/(3+1) = 0.5$, using a recursive CTE to carry EMA forward.

Ex 29. Write a recursive CTE to flatten a JSON-like nested category path stored as 'Electronics > Computers > Laptops' into one row per ancestor level for each category: (category_id, ancestor_name, level).

Ex 30. CHALLENGE: Write a single SQL statement (multiple CTEs allowed) that produces a complete dependency report for the depends_on table: package, all_transitive_deps (comma-separated), total_dep_count, max_chain_depth, is_circular (bool), and install_order (topological rank). Handle diamond dependencies and self-references correctly.

Chapter 15 · Full Answer Key

Answers 1–10 (Beginner)

Answer 1

```
WITH RECURSIVE nums AS (  
  
  SELECT 1 AS n  
  
  UNION ALL  
  
  SELECT n + 1 FROM nums WHERE n < 50  
  
)  
  
SELECT n, n * n AS square FROM nums ORDER BY n;
```

Answer 2

```
WITH RECURSIVE jan AS (  
  
  SELECT CAST('2025-01-01' AS DATE) AS dt  
  
  UNION ALL  
  
  SELECT dt + INTERVAL '1 day' FROM jan WHERE dt < '2025-01-31'  
  
)  
  
SELECT dt FROM jan;
```

Answer 3

```
WITH RECURSIVE org AS (  
  
  SELECT emp_id, name, manager_id, 0 AS depth  
  
  FROM employees WHERE manager_id IS NULL  
  
  UNION ALL  
  
  SELECT e.emp_id, e.name, e.manager_id, o.depth + 1  
  
  FROM employees e JOIN org o ON e.manager_id = o.emp_id  
  
)
```

```
SELECT depth, name FROM org ORDER BY depth, name;
```

Answer 4

```
WITH RECURSIVE reports AS (

  SELECT emp_id, name, manager_id

  FROM employees WHERE manager_id = 1 -- Sarah Chen

  UNION ALL

  SELECT e.emp_id, e.name, e.manager_id

  FROM employees e JOIN reports r ON e.manager_id = r.emp_id

)

SELECT name FROM reports;
```

Answer 5

```
WITH RECURSIVE paths AS (

  SELECT emp_id, name, manager_id,

  CAST(name AS VARCHAR(2000)) AS path

  FROM employees WHERE manager_id IS NULL

  UNION ALL

  SELECT e.emp_id, e.name, e.manager_id,

  p.path || ' / ' || e.name

  FROM employees e JOIN paths p ON e.manager_id = p.emp_id

)

SELECT name, path FROM paths ORDER BY path;
```

Answer 6

```
WITH RECURSIVE fib AS (

  SELECT 0 AS a, 1 AS b, 1 AS step
```

```

UNION ALL

SELECT b, a + b, step + 1 FROM fib WHERE step < 20

)

SELECT a AS fibonacci FROM fib ORDER BY step;

```

Answer 7

```

WITH RECURSIVE ancestry AS (

SELECT emp_id, name, manager_id, 0 AS level

FROM employees WHERE name = 'Liam Brown'

UNION ALL

SELECT p.emp_id, p.name, p.manager_id, a.level + 1

FROM employees p JOIN ancestry a ON p.emp_id = a.manager_id

)

SELECT level, name FROM ancestry WHERE level > 0 ORDER BY level;

```

Answer 8

```

WITH RECURSIVE mult AS (

SELECT 1 AS factor, 7 AS result

UNION ALL

SELECT factor + 1, 7 * (factor + 1) FROM mult WHERE factor < 12

)

SELECT '7 x ' || factor || ' = ' || result AS multiplication FROM mult;

```

Answer 9

```

WITH RECURSIVE org AS (

SELECT emp_id, name, manager_id, 0 AS depth

FROM employees WHERE manager_id IS NULL

```

```

UNION ALL

SELECT e.emp_id, e.name, e.manager_id, o.depth + 1

FROM employees e JOIN org o ON e.manager_id = o.emp_id

)

SELECT name FROM org WHERE depth = 2;

```

Answer 10

```

WITH RECURSIVE q1 AS (

SELECT CAST('2025-01-01' AS DATE) AS dt

UNION ALL

SELECT dt + INTERVAL '1 day' FROM q1 WHERE dt < '2025-03-31'

)

SELECT dt, TO_CHAR(dt, 'Day') AS day_name

FROM q1

WHERE EXTRACT(DOW FROM dt) BETWEEN 1 AND 5 -- Mon=1 Fri=5

ORDER BY dt;

```

Answers 11–20 (Intermediate)

Answer 11

```

WITH RECURSIVE subtree AS (

SELECT emp_id AS root_id, emp_id, salary

FROM employees

UNION ALL

SELECT s.root_id, e.emp_id, e.salary

FROM employees e JOIN subtree s ON e.manager_id = s.emp_id

)

```

```

SELECT root_id, m.name, SUM(salary) AS total_budget, COUNT(*) AS headcount

FROM subtree JOIN employees m ON m.emp_id = root_id

GROUP BY root_id, m.name ORDER BY total_budget DESC;

```

Answer 12

```

WITH RECURSIVE sp AS (

SELECT to_city AS city, distance AS total_dist

FROM roads WHERE from_city = 'Dubai'

UNION ALL

SELECT r.to_city, sp.total_dist + r.distance

FROM roads r JOIN sp ON r.from_city = sp.city

WHERE sp.total_dist + r.distance < 1000 -- reasonable cap

)

SELECT city, MIN(total_dist) AS shortest_dist

FROM sp GROUP BY city ORDER BY shortest_dist;

```

Answer 13

```

WITH RECURSIVE split AS (

SELECT 'alpha,beta,gamma,delta,epsilon,' AS remaining, '' AS token

UNION ALL

SELECT SUBSTRING(remaining FROM POSITION(',') + 1),

SUBSTRING(remaining FOR POSITION(',') - 1)

FROM split WHERE POSITION(',') IN remaining) > 0

)

SELECT token FROM split WHERE token <> '' ORDER BY token;

```

Answer 14

```

WITH RECURSIVE closure AS (

    SELECT package AS root, requires AS dep, 1 AS hops

    FROM depends_on

    UNION

    SELECT c.root, d.requires, c.hops + 1

    FROM closure c JOIN depends_on d ON d.package = c.dep

    WHERE c.hops < 20

)

SELECT root AS package, dep AS transitive_dependency, MIN(hops) AS min_hops

FROM closure GROUP BY root, dep ORDER BY root, min_hops;

```

Answer 15

```

WITH RECURSIVE tenure AS (

    SELECT emp_id, name, hire_date AS month_start

    FROM employees

    UNION ALL

    SELECT emp_id, name,

    (month_start + INTERVAL '1 month')::DATE

    FROM tenure

    WHERE month_start < '2025-12-01'

)

SELECT emp_id, name, month_start

FROM tenure ORDER BY emp_id, month_start;

```

Answer 16

```

WITH RECURSIVE closure AS (

```

```

SELECT package AS root, requires AS dep

FROM depends_on

UNION

SELECT c.root, d.requires

FROM closure c JOIN depends_on d ON d.package = c.dep

)

SELECT DISTINCT root AS affected_package

FROM closure WHERE dep = 'lib_a'

ORDER BY root;

```

Answer 17

```

WITH RECURSIVE cal AS (

SELECT CAST('2025-01-01' AS DATE) AS dt

UNION ALL

SELECT dt + INTERVAL '1 day' FROM cal WHERE dt < '2025-12-31'

)

SELECT dt,

TO_CHAR(dt, 'Day') AS day_name,

EXTRACT(WEEK FROM dt) AS week_number,

EXTRACT(DOW FROM dt) IN (0,6) AS is_weekend

FROM cal ORDER BY dt;

```

Answer 18

```

WITH RECURSIVE running AS (

SELECT txn_id, txn_date, amount, amount AS balance

FROM ledger WHERE txn_id = (SELECT MIN(txn_id) FROM ledger)

```

```

UNION ALL

SELECT l.txn_id, l.txn_date, l.amount, r.balance + l.amount

FROM ledger l

JOIN running r ON l.txn_id = (

SELECT MIN(txn_id) FROM ledger WHERE txn_id > r.txn_id

)

SELECT txn_id, txn_date, amount, balance FROM running ORDER BY txn_id;

```

Answer 19

```

WITH RECURSIVE org AS (

SELECT emp_id, name, manager_id, 0 AS depth

FROM employees WHERE manager_id IS NULL

UNION ALL

SELECT e.emp_id, e.name, e.manager_id, o.depth + 1

FROM employees e JOIN org o ON e.manager_id = o.emp_id

WHERE o.depth < 3

)

SELECT name FROM org WHERE depth = 3;

```

Answer 20

```

WITH RECURSIVE sp AS (

SELECT to_city AS city, distance AS dist,

CAST('Dubai -> ' || to_city AS VARCHAR(2000)) AS route

FROM roads WHERE from_city = 'Dubai'

UNION ALL

SELECT r.to_city, sp.dist + r.distance,

```

```

sp.route || ' -> ' || r.to_city

FROM roads r JOIN sp ON r.from_city = sp.city

WHERE sp.route NOT LIKE '%' || r.to_city || '%'

)

SELECT route, dist FROM sp WHERE city = 'Fujairah'

ORDER BY dist LIMIT 1;

```

Answers 21–30 (Advanced)

Answer 21

```

WITH RECURSIVE cycle_check AS (

SELECT emp_id, manager_id,

ARRAY[emp_id] AS visited,

false AS cycle

FROM employees

UNION ALL

SELECT cc.emp_id, e.manager_id,

cc.visited || e.emp_id,

e.emp_id = ANY(cc.visited)

FROM employees e

JOIN cycle_check cc ON e.emp_id = cc.manager_id

WHERE NOT cc.cycle AND cc.manager_id IS NOT NULL

)

SELECT emp_id, visited AS cycle_path

FROM cycle_check WHERE cycle;

```

Answer 22

```

-- BFS: each city visited only at minimum hop count

WITH RECURSIVE bfs AS (

SELECT to_city AS city, distance AS total_dist, 1 AS hops

FROM roads WHERE from_city = 'Dubai'

UNION

SELECT r.to_city, bfs.total_dist + r.distance, bfs.hops + 1

FROM roads r JOIN bfs ON r.from_city = bfs.city

WHERE bfs.hops < 10

)

SELECT city, MIN(hops) AS hops, MIN(total_dist) AS shortest_dist

FROM bfs GROUP BY city ORDER BY shortest_dist;

```

Answer 23

```

WITH RECURSIVE

candidates AS (

SELECT 2 AS n UNION ALL SELECT n + 1 FROM candidates WHERE n < 200

),

primes AS (

SELECT n FROM candidates c

WHERE NOT EXISTS (

SELECT 1 FROM candidates d

WHERE d.n BETWEEN 2 AND FLOOR(SQRT(c.n))

AND c.n % d.n = 0

)

)

```

```
SELECT n AS prime FROM primes ORDER BY n;
```

Answer 24

```
-- Org weight = SUM(1 / 2^relative_depth) for all nodes in subtree

WITH RECURSIVE subtree AS (

  SELECT emp_id AS root_id, emp_id AS node_id, 0 AS rel_depth

  FROM employees

  UNION ALL

  SELECT s.root_id, e.emp_id, s.rel_depth + 1

  FROM employees e JOIN subtree s ON e.manager_id = s.node_id

)

SELECT root_id, m.name,

SUM(1.0 / POWER(2, rel_depth)) AS org_weight

FROM subtree JOIN employees m ON m.emp_id = root_id

GROUP BY root_id, m.name

ORDER BY org_weight DESC;
```

Answer 25

```
WITH RECURSIVE org AS (

  SELECT emp_id, name, manager_id, salary, dept,

  0 AS depth,

  CAST(name AS VARCHAR(2000)) AS path

  FROM employees WHERE manager_id IS NULL

  UNION ALL

  SELECT e.emp_id, e.name, e.manager_id, e.salary, e.dept,

  o.depth + 1, o.path || ' > ' || e.name
```

```

FROM employees e JOIN org o ON e.manager_id = o.emp_id

),

subtree_stats AS (

SELECT s.root_id, COUNT(*) AS headcount, SUM(salary) AS total_sal

FROM (

WITH RECURSIVE sub AS (

SELECT emp_id AS root_id, emp_id, salary FROM employees

UNION ALL

SELECT s.root_id, e.emp_id, e.salary

FROM employees e JOIN sub s ON e.manager_id = s.emp_id

) SELECT * FROM sub

) s GROUP BY s.root_id

)

SELECT o.emp_id, o.name, o.depth, o.path,

ss.headcount, ss.total_sal,

COUNT(DISTINCT hi.salary) + 1 AS salary_rank_in_depth

FROM org o

JOIN subtree_stats ss ON ss.root_id = o.emp_id

LEFT JOIN org hi ON hi.depth = o.depth AND hi.salary > o.salary

GROUP BY o.emp_id, o.name, o.depth, o.path, ss.headcount, ss.total_sal

ORDER BY o.depth, salary_rank_in_depth;

```

Answer 26

```

-- Topological sort: tasks with no unresolved deps come first

WITH RECURSIVE topo AS (

```

```
-- Anchor: tasks with no dependency

SELECT task_id, task_name, depends_on_task_id, 0 AS sort_order

FROM tasks WHERE depends_on_task_id IS NULL

UNION ALL

SELECT t.task_id, t.task_name, t.depends_on_task_id, tp.sort_order + 1

FROM tasks t

JOIN topo tp ON t.depends_on_task_id = tp.task_id

)

SELECT sort_order, task_name FROM topo ORDER BY sort_order, task_id;
```

Answer 27

```
-- All routes Dubai -> Al Ain (no cycle)

WITH RECURSIVE routes AS (

SELECT to_city AS city, distance AS total_dist,

CAST('Dubai -> ' || to_city AS VARCHAR(2000)) AS route

FROM roads WHERE from_city = 'Dubai'

UNION ALL

SELECT r.to_city,

rt.total_dist + r.distance,

rt.route || ' -> ' || r.to_city

FROM roads r

JOIN routes rt ON r.from_city = rt.city

WHERE rt.route NOT LIKE '%' || r.to_city || '%' -- cycle guard

AND rt.total_dist + r.distance < 1000 -- distance cap

)
```

```
SELECT route, total_dist

FROM routes WHERE city = 'Al Ain'

ORDER BY total_dist;
```

Answer 28

```
-- EMA(t) = amount * k + EMA(t-1) * (1-k), k = 0.5

WITH RECURSIVE ema AS (

SELECT txn_id, txn_date, amount,

amount::DECIMAL(14,4) AS ema_val

FROM ledger WHERE txn_id = (SELECT MIN(txn_id) FROM ledger)

UNION ALL

SELECT l.txn_id, l.txn_date, l.amount,

ROUND(l.amount * 0.5 + e.ema_val * 0.5, 4)

FROM ledger l

JOIN ema e ON l.txn_id = (

SELECT MIN(txn_id) FROM ledger WHERE txn_id > e.txn_id

)

)

SELECT txn_id, txn_date, amount, ema_val FROM ema ORDER BY txn_id;
```

Answer 29

```
-- categories table: (cat_id, path) e.g. 'Electronics > Computers > Laptops'

WITH RECURSIVE flatten AS (

SELECT cat_id,

path AS remaining,

'' AS ancestor,
```

```

0 AS level

FROM categories

UNION ALL

SELECT cat_id,

CASE WHEN POSITION(' > ' IN remaining) > 0

THEN SUBSTRING(remaining FROM POSITION(' > ' IN remaining) + 3)

ELSE '' END,

CASE WHEN POSITION(' > ' IN remaining) > 0

THEN SUBSTRING(remaining FOR POSITION(' > ' IN remaining) - 1)

ELSE remaining END,

level + 1

FROM flatten

WHERE remaining <> ''

)

SELECT cat_id, ancestor AS ancestor_name, level

FROM flatten WHERE ancestor <> ''

ORDER BY cat_id, level;

```

Answer 30

```

-- CHALLENGE: Full dependency report

WITH RECURSIVE

closure AS (

SELECT package AS root, requires AS dep, 1 AS hops

FROM depends_on

UNION

```

```

SELECT c.root, d.requires, c.hops + 1

FROM closure c JOIN depends_on d ON d.package = c.dep

WHERE c.hops < 30

),

cycle_check AS (

SELECT DISTINCT root AS package, true AS is_circular

FROM closure WHERE root = dep

),

topo AS (

SELECT package, 0 AS rank

FROM depends_on GROUP BY package

HAVING COUNT(requires) = 0

UNION ALL

SELECT d.package, t.rank + 1

FROM depends_on d JOIN topo t ON d.requires = t.package

)

SELECT

c.root AS package,

STRING_AGG(DISTINCT c.dep, ',' ORDER BY c.dep) AS all_transitive_deps,

COUNT(DISTINCT c.dep) AS total_dep_count,

MAX(c.hops) AS max_chain_depth,

COALESCE(cc.is_circular, false) AS is_circular,

COALESCE(MAX(t.rank), 0) AS install_order

FROM closure c

```

```
LEFT JOIN cycle_check cc ON cc.package = c.root  
  
LEFT JOIN topo t ON t.package = c.root  
  
GROUP BY c.root, cc.is_circular  
  
ORDER BY install_order, package;
```

— End of Guide —

Work through every exercise without hints first. Profile each query on real data. That gap between 'it runs' and 'it scales' is what separates a senior developer from everyone else.